

Djezzy National SOC Platform

Comprehensive Deployment Documentation

Hardware Architecture Guide | Operations Manual | Quick-Start Cheat Sheet
Wazuh + TheHive Integration Setup

Platform Scale	Technology Stack	Deployment Model
39 Microservices	Next.js 16 / React 19 / TypeScript	100% On-Premises
15 Security Tools	PostgreSQL 16 / Redis 7 / Kafka	Air-Gap Capable
22 Servers (768GB RAM)	Docker / Kubernetes / Helm	Zero Cloud Dependencies
50B+ Events/Year	Prisma ORM / Grafana / Prometheus	Primary + DR Site

Document Version: 1.0
Generated: 2026-08-03 15:42 UTC
Classification: Internal Operations Document
Repository: <https://github.com/LAIDOUDI33/NetOP>

Table of Contents

PART I: Hardware Architecture Guide	3
1.1 Primary Site Infrastructure (14 Servers)	3
1.2 Disaster Recovery Site (8 Servers)	4
1.3 AI & Geomarketing Resource Allocation	5
1.4 Network Topology & Connectivity	6
PART II: Operations Manual	7
2.1 Daily Operational Procedures	7
2.2 Monitoring & Alerting	8
2.3 Backup & Recovery Procedures	9
2.4 Security Patch Management	10
2.5 Incident Response Workflow	11
PART III: Quick-Start Cheat Sheet	12
3.1 Pre-Deployment Checklist	12
3.2 Docker Compose Deployment	13
3.3 Kubernetes Deployment	14
3.4 Environment Variables Reference	15
3.5 Common Commands Quick Reference	16
PART IV: Wazuh + TheHive Integration Setup	17
4.1 Integration Architecture Overview	17
4.2 Wazuh Configuration for TheHive	18
4.3 TheHive Receiver Configuration	19
4.4 Automated Case Creation Workflow	20
4.5 Testing & Validation Procedures	21
Appendix A: Port Mapping Reference	22
Appendix B: Troubleshooting Guide	23

PART I: Hardware Architecture Guide

This section provides comprehensive hardware specifications for the Djezzy National SOC Platform deployment. The architecture follows a dual-site design with a primary production site and a disaster recovery site, ensuring business continuity and high availability for critical security operations. All infrastructure is designed for 100% on-premises deployment with zero cloud dependencies, meeting strict data sovereignty requirements while supporting air-gap capable operation scenarios.

1.1 Primary Site Infrastructure (14 Servers)

The primary site houses the core operational infrastructure for the SOC platform, comprising 14 dedicated servers organized into functional clusters. Each server role is carefully defined to optimize resource utilization while maintaining redundancy for critical components. The primary site handles all real-time security monitoring, event processing, and analyst operations during normal operations.

Server ID	Role	Specification	Services Hosted	Network
SOC-DB-01	Primary Database	2x Intel Xeon Gold 6348 (56 cores) 512GB DDR4 ECC 8TB NVMe RAID10 10Gbps x4 bonded	PostgreSQL 16 (Primary) PgBouncer Connection Pooler	soc-backend
SOC-DB-02	Database Replica	2x Intel Xeon Gold 6348 (56 cores) 512GB DDR4 ECC 8TB NVMe RAID10 10Gbps x4 bonded	PostgreSQL 16 (Hot Standby) WAL Streaming Replica	soc-backend
SOC-CACHE-01	Cache & Session Store	Intel Xeon Silver 4314 (32 cores) 256GB DDR4 ECC 4TB NVMe RAID1 10Gbps x2 bonded	Redis 7 (Master) Redis Sentinel Leader	soc-backend
SOC-CACHE-02	Cache Replica	Intel Xeon Silver 4314 (32 cores) 256GB DDR4 ECC 4TB NVMe RAID1 10Gbps x2 bonded	Redis 7 (Replica) Redis Sentinel Node	soc-backend
SOC-ES-01	Elasticsearch Master	2x Intel Xeon Gold 6348 (56 cores) 512GB DDR4 ECC 12TB NVMe RAID10 25Gbps x2	Elasticsearch 8.x (Master) Kibana Visualizations	soc-backend
SOC-ES-02	Elasticsearch Data 1	2x Intel Xeon Gold 6348 (56 cores) 512GB DDR4 ECC 24TB NVMe RAID10 25Gbps x2	Elasticsearch 8.x (Data-Hot) Wazuh Indexer	soc-backend
SOC-ES-03	Elasticsearch Data 2	2x Intel Xeon Gold 6348 (56 cores) 512GB DDR4 ECC 24TB NVMe RAID10 25Gbps x2	Elasticsearch 8.x (Data-Warm) Log Archive Indexing	soc-backend

SOC-KAFKA-01	Event Bus Broker 1	Intel Xeon Gold 5318Y (24 cores) 128GB DDR4 ECC 4TB NVMe RAID1 25Gbps x2	Apache Kafka (Broker-1) ZooKeeper Ensemble	soc-events
SOC-KAFKA-02	Event Bus Broker 2	Intel Xeon Gold 5318Y (24 cores) 128GB DDR4 ECC 4TB NVMe RAID1 25Gbps x2	Apache Kafka (Broker-2) ZooKeeper Ensemble	soc-events
SOC-KAFKA-03	Event Bus Broker 3	Intel Xeon Gold 5318Y (24 cores) 128GB DDR4 ECC 4TB NVMe RAID1 25Gbps x2	Apache Kafka (Broker-3) ZooKeeper Ensemble	soc-events
SOC-SIEM-01	SIEM & EDR Core	2x Intel Xeon Gold 6348 (56 cores) 384GB DDR4 ECC 8TB NVMe RAID10 25Gbps x2	Wazuh Manager GRR Server Osquery Fleet	soc-backend
SOC-NSM-01	Network Security Monitor	2x Intel Xeon Gold 6348 (56 cores) 256GB DDR4 ECC 8TB NVMe RAID10 100Gbps (SPAN)	Suricata IDS Zeek Controller Arkime PCAP	soc-events
SOC-SOAR-01	SOAR & Threat Intel	2x Intel Xeon Gold 5318Y (48 cores) 256GB DDR4 ECC 8TB NVMe RAID10 10Gbps x4	TheHive Project Cortex Analyzers MISP OpenCTI	soc-backend
SOC-APP-01	Application & API	2x Intel Xeon Gold 5318Y (48 cores) 256GB DDR4 ECC 4TB NVMe RAID10 10Gbps x4	SOC Platform (Next.js) Kong API Gateway Nginx Reverse Proxy	soc-frontend

Primary Site Resource Summary: Total of ~140 vCPUs (with hyperthreading: ~280 threads), ~4,864GB RAM, ~125TB NVMe storage across 14 servers. Network capacity ranges from 10Gbps for management traffic to 100Gbps for SPAN/mirror ports on the NSM server. All database and cache layers implement synchronous replication with automatic failover capabilities.

1.2 Disaster Recovery Site (8 Servers)

The disaster recovery (DR) site provides geographic redundancy and business continuity capability for the SOC platform. Located at a minimum distance of 50km from the primary site to avoid regional disaster overlap, the DR site maintains warm standby instances that can assume full operational load within the defined RTO (Recovery Time Objective) of 4 hours and RPO (Recovery Point Objective) of 1 hour for transactional data.

Server ID	Role	Specification	Services Hosted	Replication Mode
DR-DB-01	DR Database	2x Intel Xeon Gold 5318Y (48 cores) 384GB DDR4 ECC 8TB NVMe RAID10 10Gbps x4	PostgreSQL 16 (DR Standby) PgBouncer DR Instance	Async Streaming (RPO: 1hr)

DR-CACHE-01	DR Cache Layer	Intel Xeon Silver 4314 (32 cores) 128GB DDR4 ECC 2TB NVMe RAID1 10Gbps x2	Redis 7 (DR Replica) Sentinel DR Node	Async Replication (RPO: 5min)
DR-ES-01	DR Elasticsearch	2x Intel Xeon Gold 5318Y (48 cores) 256GB DDR4 ECC 12TB NVMe RAID10 10Gbps x4	Elasticsearch 8.x (DR) Cross-Cluster Replication	CCR (RPO: 15min)
DR-KAFKA-01	DR Event Bus	Intel Xeon Gold 5318Y (24 cores) 64GB DDR4 ECC 2TB NVMe RAID1 10Gbps x2	Kafka MirrorMaker DR Consumer Group	Mirror (RPO: 5min)
DR-SIEM-01	DR SIEM Stack	2x Intel Xeon Gold 5318Y (48 cores) 256GB DDR4 ECC 4TB NVMe RAID10 10Gbps x4	Wazuh Manager (DR) GRR Server (DR)	Config Sync (Event Replay)
DR-SOAR-01	DR SOAR Stack	Intel Xeon Gold 5318Y (24 cores) 128GB DDR4 ECC 4TB NVMe RAID10 10Gbps x2	TheHive (DR) Cortex (DR) MISP (DR)	DB Replication (Config Sync)
DR-MON-01	DR Monitoring	Intel Xeon Silver 4314 (32 cores) 64GB DDR4 ECC 2TB NVMe RAID1 10Gbps x2	Prometheus (DR) Grafana (DR) Alertmanager (DR)	Federation (RT: 30s)
DR-APP-01	DR Application	2x Intel Xeon Gold 5318Y (48 cores) 128GB DDR4 ECC 2TB NVMe RAID10 10Gbps x4	SOC Platform (DR) Kong (DR) Nginx (DR)	Container Sync (Config Sync)

DR Site Resource Summary: Total of ~80 vCPUs (~160 threads), ~1,280GB RAM, ~36TB NVMe storage across 8 servers. The DR site operates at approximately 40% of primary site capacity, sufficient to maintain essential security monitoring functions during failover scenarios. Full capacity restoration through auto-scaling or manual provisioning can be achieved within 8 hours of declaring a disaster event.

1.3 AI & Geomarketing Resource Allocation

The SOC platform incorporates advanced AI-driven analytics and geomarketing capabilities that require specialized resource allocation. These workloads are distributed across existing infrastructure with dedicated resource pools to ensure performance isolation and predictable response times for both real-time security operations and batch analytical processing tasks.

1.3.1 AI/ML Service Resource Requirements

Service Component	CPU Allocation	Memory	GPU/Accelerator	Storage	Host Server
AI Automation Engine	16 vCPU reserved	64GB RAM	Optional: NVIDIA T4 (16GB)	500GB SSD (models)	SOC-APP-01
Threat Intelligence NLP	8 vCPU reserved	32GB RAM	None (CPU inference)	200GB SSD (corpus)	SOC-SOAR-01

Anomaly Detection ML	12 vCPU reserved	48GB RAM	Optional: NVIDIA A10 (24GB)	300GB SSD (training)	SOC-SIEM-01
Behavioral Analytics	8 vCPU reserved	32GB RAM	None (CPU inference)	150GB SSD (profiles)	SOC-ES-02
Predictive Alert Scoring	6 vCPU reserved	24GB RAM	None (CPU inference)	100GB SSD (models)	SOC-KAFKA-01
Natural Language Query	4 vCPU reserved	16GB RAM	None (CPU inference)	50GB SSD (embeddings)	SOC-APP-01

1.3.2 Geomarketing & Location Analytics Resources

Component	Purpose	Resource Requirement	Data Volume	Update Frequency
GeoIP Enrichment	Real-time IP geolocation lookup	4 vCPU, 16GB RAM, 100GB SSD	MaxMind DB (~200MB)	Weekly
Tower/Cell Mapping	Subscriber location via cell towers	8 vCPU, 32GB RAM, 500GB SSD	50K tower records	Daily
Heat Map Engine	Security incident visualization	4 vCPU, 16GB RAM, 200GB SSD	Aggregated events	Real-time
Coverage Analysis	Network coverage gap detection	6 vCPU, 24GB RAM, 300GB SSD	CDR + GIS data	Hourly
Location Intelligence	Pattern analysis by region	8 vCPU, 32GB RAM, 400GB SSD	Historical + live	On-demand

1.4 Network Topology & Connectivity

The SOC platform network architecture implements defense-in-depth principles with four isolated network segments, each serving specific functional purposes. Network segmentation prevents lateral movement, contains potential breaches, and ensures compliance with telecommunications security regulations. All inter-network communication flows through controlled gateways with mandatory authentication and encryption.

Network Name	CIDR Range	Purpose	VLAN ID	Security Level	Bandwidth
soc-frontend	10.0.1.0/24	User-facing services, API gateway, web portal	VLAN 101	DMZ	10Gbps
soc-backend	10.0.2.0/23	Application servers, databases, cache layer	VLAN 102-103	Internal	25Gbps
soc-events	10.0.4.0/22	Event streaming, NSM tools, packet capture	VLAN 104-107	Restricted	100Gbps
soc-monitoring	10.0.8.0/24	Monitoring, logging, metrics collection	VLAN 108	Management	10Gbps

PART II: Operations Manual

This Operations Manual establishes standardized procedures for day-to-day management of the Djezzy SOC Platform. It covers routine operational tasks, monitoring protocols, backup procedures, patch management workflows, and incident response coordination. All procedures are designed to ensure consistent, reliable operation while maintaining security posture and regulatory compliance requirements specific to Algerian telecommunications operators.

2.1 Daily Operational Procedures

Daily operations follow a structured shift-handover protocol with documented checklists for each operational period. The SOC operates on a 24/7 schedule with three 8-hour shifts, each staffed by a minimum of two Level 1 analysts and one Level 2 senior analyst. Shift handovers must include verbal briefing plus written documentation in the shift log system covering ongoing incidents, pending tasks, and any anomalies observed during the previous shift.

2.1.1 Morning Shift Start (08:00 UTC+1)

Task	Description	Frequency	Responsible Role	SLA Target
System Health Check	Verify all 39 services showing HEALTHY status on dashboard	Every shift	L1 Analyst	15 min completion
Queue Review	Process overnight alerts, triage by severity	Every shift	L1 Analyst	30 min review
Backup Verification	Confirm nightly backups completed successfully	Daily	L2 Senior	Check within 1hr
Capacity Review	Review CPU/Memory/Disk utilization trends	Daily	L2 Senior	Document anomalies
Threat Feed Update	Verify MISP/OpenCTI feeds updated in last 24h	Daily	L1 Analyst	Flag if stale >48h
Ticket Aging	Escalate tickets open >72hr without progress	Daily	L2 Senior	Escalate by 09:00

2.1.2 End-of-Day Procedures (20:00 UTC+1)

Task	Details	Documentation Required
Shift Summary Report	Compile metrics: alerts processed, incidents opened/closed, escalations	Ad hoc summary + analyst notes
Handover Briefing	Verbal handover to night shift with priority items highlighted	Handover checklist signed
Pending Work Log	Document all incomplete tasks with status and next steps	Ticket updates required
Anomaly Log	Record any unusual patterns or behaviors requiring follow-up	Open incident ticket if needed
Resource Utilization	Snapshot current resource usage for trend analysis	Automated dashboard export

2.2 Monitoring & Alerting Framework

The monitoring framework employs a multi-layer approach combining infrastructure metrics, application performance monitoring, and security-specific alerting. Prometheus serves as the primary metrics collection engine with 15-second scrape intervals for critical infrastructure and 30-second intervals for

standard services. Grafana provides unified dashboards with role-based access controlling visibility into different operational tiers based on analyst clearance level.

2.2.1 Critical Alerts (Immediate Response Required)

Alert Name	Condition	Threshold	Response Time	Escalation Path
Service Down	Any of 39 services status != RUNNING	Immediate	< 5 minutes	On-call -> Manager -> Director
Database Connection Fail	PostgreSQL connection pool exhausted	> 95% usage	< 10 minutes	DBA -> Infrastructure Lead
EPS Spike	Events per second exceeds baseline	> 2x normal (100K EPS)	< 15 minutes	L1 -> L2 -> Incident Commander
Disk Critical	Storage utilization on any node	> 90%	< 30 minutes	Ops -> Storage Team
Authentication Failure	Failed login attempts spike	> 50/min per source	< 5 minutes	Security -> IR Team
Replication Lag	PostgreSQL or Kafka replication delay	> 5 minutes	< 15 minutes	DBA -> Infrastructure Lead

2.2.2 Dashboard Inventory

Dashboard Name	Purpose	Refresh Rate	Access Level	Key Panels
SOC Overview	Executive summary of all systems	30s	All roles	Service health, EPS, queue depth
SIEM Operations	Wazuh alerts and investigations	15s	L1+	Alert volume, top rules, MITRE map
Infrastructure	Server and container health	30s	L2+	CPU, memory, disk, network I/O
Event Pipeline	Kafka and data flow metrics	15s	L2+	Consumer lag, throughput, errors
Threat Intelligence	MISP/OpenCTI feed status	5min	L1+	Feed health, IOC counts, last update
Geospatial	Security events by location	60s	L1+	Heat map, regional breakdown, towers

2.3 Backup & Recovery Procedures

Data protection strategy implements the 3-2-1 backup rule: three copies of data, two different media types, one copy off-site (DR location). Backups are encrypted at rest using AES-256 encryption with keys managed through HashiCorp Vault. Recovery procedures are tested quarterly during scheduled maintenance windows to validate RPO/RTO targets and identify any procedure gaps before actual disaster scenarios occur.

Data Type	Backup Method	Frequency	Retention	RPO Target	Storage Location
PostgreSQL DB	pg_basebackup + WAL archiving	Continuous (WAL) Full: Daily	30 days 7 days	1 hour	NAS Primary + DR sync
Elasticsearch Indices	Snapshot to S3-compatible	Incremental: Hourly Full: Weekly	90 days 30 days	1 hour	Object Storage (local + DR)
Redis Cache	RDB snapshot + AOF	RDB: Hourly AOF: Continuous	24 hours 1 hour	5 minutes	Local NVMe + replica sync
Kafka Topics	MirrorMaker to DR + S3	Real-time mirror Batch archive	7 days 90 days	5 minutes	DR Cluster + Object Storage
Configuration	Git repository backup	On change + Daily push	365 days	Instant	GitLab + off-site backup
Container Images	Registry replication	On push + Daily sync	Version history	Instant	Harbor Registry (primary + DR)

2.4 Security Patch Management

Security patch management follows a risk-based prioritization matrix aligned with CVSS scores and threat intelligence context. Critical patches (CVSS 9.0+) must be deployed within 72 hours of vendor release, high severity (CVSS 7.0-8.9) within 14 days, and medium severity (CVSS 4.0-6.9) within 30 days. All patches undergo testing in a staging environment mirroring production configuration before deployment to minimize operational risk.

Phase	Activities	Duration	Approvals Required	Rollback Plan
1. Assessment	Identify affected systems, assess exploitability, 48 hours	48 hours	Security Team Lead	Document impact scope
2. Testing	Deploy to staging environment, run regression tests, validate functionality, 24 hours	24 hours	QA Engineer	Staging snapshot restore
3. Scheduling	Coordinate maintenance window, notify stakeholders, 24 hours	24 hours	Operations Manager	Schedule rollback window
4. Deployment	Apply patches using rolling updates, monitor health checks, verify services, 12 hours	12 hours	Service Advisory Board	Automatic rollback on failure
5. Validation	Run post-deployment checks, confirm vulnerability remediation, 24 hours	24 hours	Security Team Lead	Previous version available 7 days

2.5 Incident Response Workflow

The incident response workflow integrates directly with TheHive case management system and follows the NIST Incident Response framework adapted for telecommunications SOC environments. All security incidents are classified according to severity levels that determine response timelines, escalation paths, and communication requirements. The workflow supports both automated response playbooks for common scenarios and manual investigation procedures for complex or novel threats requiring human judgment and forensic analysis expertise.

Severity	Definition	Response SLA	Escalation	Communication	Example
P1 - Critical	Active breach, data exfiltration, ransomware, 15 min acknowledgement	15 min acknowledgement 1 hour containment	IR Commander -> CISO	Executive + Regulatory	APT with C2 established
P2 - High	Confirmed compromise attempt, DoS, malware, 30 min acknowledgement	30 min acknowledgement 4 hours containment	IR Team Lead -> CISO	Management + Affected Teams	Phishing with credential theft
P3 - Medium	Suspicious activity, policy violation, 2 hour acknowledgement	2 hour acknowledgement 24 hours resolution	L2 Senior -> IR Team Lead	SOC team + Department	Botnet force attack detected
P4 - Low	Informational, benign false positive, 4 hour acknowledgement	4 hour acknowledgement 72 hours resolution	L1 Analyst -> L2 Senior	SOC team only	Single failed login from VPN

PART III: Quick-Start Cheat Sheet

This Quick-Start Cheat Sheet provides rapid reference information for deployment teams installing or maintaining the Djezzy SOC Platform. Commands are optimized for bash shell environments running Ubuntu 22.04 LTS or RHEL 9. All commands assume appropriate sudo privileges and network connectivity to package repositories (or local mirrors for air-gapped deployments). Keep this section accessible during initial deployment and troubleshooting sessions.

3.1 Pre-Deployment Checklist

Category	Check Item	Command / Verification	Required Value
OS	Operating System version	cat /etc/os-release grep PRETTY_NAME	Ubuntu 22.04 LTS / RHEL 9.x
OS	Kernel version (for BPF/ebpf)	uname -r	>= 5.15 (prefer 6.x)
Hardware	Total memory available	free -h grep Mem:	Per server spec (min 64GB)
Hardware	CPU cores available	nproc	Per server spec (min 16)
Storage	Disk space for /var/lib/docker	df -h /var/lib/docker	Min 500GB free
Network	DNS resolution working	dig github.com +short	Returns IP address
Network	Connectivity to registry	curl -I https://registry-1.docker.io/v2/	HTTP 401 or 200
Security	SELinux/AppArmor status	getenforce / aa-status	Enforcing (document exception)
Security	Firewall rules reviewed	iptables -L -n head -20	Ports documented below open
Docker	Docker installed	docker --version	>= 24.0
Docker	Docker Compose plugin	docker compose version	>= 2.20
Docker	Daemon running	systemctl is-active docker	active
Prerequisites	Git installed	git --version	>= 2.40
Prerequisites	Make installed	make --version	>= 4.3
Repository	Code cloned	ls -la docker-compose.prod.yml	File exists

3.2 Docker Compose Deployment

The Docker Compose deployment method is recommended for initial installations, development environments, and single-site deployments. Production deployments should consider Kubernetes for improved orchestration, scaling, and self-healing capabilities. The following commands assume you are in the project root directory containing the docker-compose.prod.yml file cloned from the official repository.

3.2.1 Initial Deployment Sequence

ries for persistent volumes
s,elasticsearch,kafka,wazuh,suricata,thehive,misp,grafana}

te and configure

ic values (see Section 3.4)

ownload to reduce downtime)

```
prod.yml pull
```

ervices first (databases, cache, message bus)

```
prod.yml up -d postgresql redis-master pgbouncer \
```

```
h-master kafka-broker-1 kafka-broker-2 kafka-broker-3 zookeeper-1 zookeeper-2 zookeeper-3
```

e readiness (approx 2-3 minutes)

```
cker-compose.prod.yml ps
```

vices

```
prod.yml up -d wazuh-manager wazuh-indexer grr-server \
```

```
thehive cortex misp opencti openvas defectdojo
```

last

```
prod.yml up -d kong nginx soc-platform
```

healthy

```
prod.yml ps --format "table {{.Name}}\t{{.Status}}"
```

3.2.2 Common Operations Commands

Operation	Command
View service logs (follow)	<code>docker compose -f docker-compose.prod.yml logs -f --tail=100 <service-name></code>
Restart single service	<code>docker compose -f docker-compose.prod.yml restart <service-name></code>
Scale a service	<code>docker compose -f docker-compose.prod.yml up -d --scale <service-name>=<replicas></code>
Stop all services	<code>docker compose -f docker-compose.prod.yml down</code>
Stop and remove volumes	<code>docker compose -f docker-compose.prod.yml down -v</code>
Check resource usage	<code>docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"</code>
Execute command in container	<code>docker compose -f docker-compose.prod.yml exec <service-name> <command></code>
View network connections	<code>docker network inspect soc-frontend soc-backend soc-events soc-monitoring</code>

3.3 Kubernetes Deployment (Helm Charts)

For production deployments requiring high availability, automatic scaling, and advanced orchestration features, the SOC platform provides Helm charts for Kubernetes deployment. The following procedures assume a working Kubernetes cluster (version 1.28+) with kubectl configured and Helm 3.x installed. The charts support both standalone installation and dependency-based deployment ordering.

ory

```
charts.djezzy.dz/soc
```

```
cies first
```

```
c/postgresql -n soc-infrastructure --create-namespace \
```

```
8Ti,replica.enabled=true
```

```
is -n soc-infrastructure \
```

```
.memory=256Gi,sentinel.enabled=true
```

```
ka -n soc-events --create-namespace \
```

```
keeper.replicaCount=3
```

```
-soc/elasticsearch -n soc-infrastructure \
```

```
a.replicaCount=2,data.persistence.size=24Ti
```

```
uh -n soc-security --create-namespace
```

```
suricata -n soc-security
```

```
hehive -n soc-security
```

```
rtex -n soc-security
```

```
-n soc-security
```

```
-n soc-frontend --create-namespace
```

```
soc/soc-platform -n soc-frontend \
```

```
ess.hostname=soc.djezzy.dz
```

```
cture,soc-events,soc-security,soc-frontend
```

3.4 Environment Variables Reference

The following environment variables must be configured in the `.env` file before deployment. Values shown are examples only; replace them with production-appropriate credentials, certificates, and connection strings. Never commit the `.env` file to version control; use secrets management (HashiCorp Vault, Kubernetes Secrets) for sensitive values in production environments.

Variable Name	Example Value	Description	Sensitive
NODE_ENV	production	Application environment mode	No
TZ	Africa/Algiers	Timezone for all containers	No
POSTGRES_HOST	postgresql	PostgreSQL service hostname	No
POSTGRES_PORT	5432	PostgreSQL port	No
POSTGRES_DB	soc_platform	Database name	No
POSTGRES_USER	soc_admin	Database username	Yes
POSTGRES_PASSWORD	<generated-strong-password>	Database password (min 32 chars)	Yes
REDIS_HOST	redis-master	Redis master hostname	No

REDIS_PORT	6379	Redis port	No
REDIS_PASSWORD	<generated-strong-password>	Redis auth password	Yes
ELASTICSEARCH_HOSTS	http://elasticsearch-master:9200	ES cluster endpoints	No
ELASTIC_PASSWORD	<generated-strong-password>	ES built-in user password	Yes
KAFKA_BOOTSTRAP_SERVERS	kafka-broker-1:9092,kafka-broker-2:9092,kafka-broker-3:9092	Kafka broker endpoints	No
WAZUH_API_URL	https://wazuh-manager:55000	Wazuh API endpoint	No
WAZUH_API_USERNAME	wazuh-wui	Wazuh API username	Yes
WAZUH_API_PASSWORD	<generated-strong-password>	Wazuh API password	Yes
THEHIVE_URL	http://thehive:9000	TheHive instance URL	No
THEHIVE_API_KEY	<generated-api-key>	TheHive API key (min 64 chars)	Yes
CORTEX_URL	http://cortex:9001	Cortex analyzer URL	No
MISP_URL	https://misp:443	MISP instance URL	No
MISP_API_KEY	<generated-api-key>	MISP authorization key	Yes
OPENCTI_URL	http://opencti:8080	OpenCTI instance URL	No
OPENCTI_API_KEY	<generated-api-key>	OpenCTI API key	Yes
GRAFANA_ADMIN_USER	admin	Grafana admin username	Yes
GRAFANA_ADMIN_PASSWORD	<generated-strong-password>	Grafana admin password	Yes
KONG_PG_HOST	postgresql	Kong database host	No
KONG_PG_PASSWORD	<generated-strong-password>	Kong database password	Yes
JWT_SECRET	<random-256-bit-hex>	JWT signing secret	Yes
ENCRYPTION_KEY	<random-256-bit-hex>	Data-at-rest encryption key	Yes

PART IV: Wazuh + TheHive Integration Setup

This section provides detailed implementation guidance for integrating Wazuh SIEM with TheHive case management platform to enable automated incident creation, alert enrichment, and streamlined security operations workflows. The integration uses Wazuh's active response mechanism combined with TheHive's REST API to automatically create cases for high-severity alerts, attach relevant context (IOCs, affected assets, timeline), and optionally trigger Cortex analyzers for automated threat intelligence enrichment.

4.1 Integration Architecture Overview

The Wazuh-TheHive integration follows an event-driven architecture where Wazuh alerts matching predefined criteria are forwarded to TheHive via a webhook or custom integration script. The integration layer sits between the two systems, performing alert filtering, deduplication, field mapping, and enrichment before creating TheHive cases. This design ensures that only actionable, high-fidelity alerts reach the case management system while reducing noise and analyst fatigue from low-priority informational events.

Component	Technology	Function	Configuration Location
Alert Source	Wazuh Manager	Generates security alerts from agents, logs, and network traffic	<code>/var/ossec/etc/ossec.conf</code>
Integration Script	Python (integration_helper.py)	Filters alerts, maps fields, calls TheHive API	<code>/var/ossec/integrations/thehive.py</code>
Case Management	TheHive Project	Stores cases, observables, tasks for analysts	<code>TheHive application.conf</code>
Analysis Engine	Cortex (optional)	Enriches IOCs with threat intel feeds	Cortex analyzer configurations
Message Queue	Apache Kafka (optional)	Decouples alert generation from case creation	<code>Kafka topic: wazuh-alerts</code>

4.2 Wazuh Configuration for TheHive Integration

Configure Wazuh to forward alerts to TheHive by modifying the main `ossec.conf` file to include the integration definition and setting up alert rules that trigger the integration. The configuration below shows the minimal setup required; customize rule IDs and severity thresholds based on your organization's risk tolerance and operational capacity for handling incoming cases.

4.2.1 Integration Definition in `ossec.conf`

```
block in /var/ossec/etc/ossec.conf -->
  point -->
    -->
  </hook_url>
  e in TheHive UI) -->
  _HERE</api_key>
  severity >= 10 (adjust as needed) -->
```

```

e (comma-separated) -->
5512,5515,5540,5550,5553,5560,5570</rule_id>
restrict to specific agent groups) -->
ritical_servers</group>
on (group similar alerts) -->
mat>
_alerts_per_group>
eout>

```

4.2.2 Custom Rules for TheHive Triggering

Define custom rules in `/var/ossec/etc/rules/local_rules.xml` to categorize alerts that should automatically create TheHive cases. The example rules below cover common attack patterns; extend these based on your threat model and historical incident data. Each rule should have a clear description that becomes part of the case title in TheHive for analyst context.

```

migration -->
>

e force detected -->
xsize="2048">

ailure">^.*$</field>
entication attack detected from srcip: ${src.ip}</description>
brute_force,thehive_critical</group>

ck (SQL injection pattern) -->
xsize="4096">

(union.*select|select.*from|insert.*into)</field>
jection attack on ${data.url}</description>
hive_critical</group>

ication on critical path -->
xsize="1024">

?:/etc|/bin|/usr/bin|/usr/sbin)</field>
file modified: ${syscheck.path}</description>
high</group>

rootcheck -->
xsize="2048">

```

```
re indicator detected: ${rootcheck.malware}</description>
e_critical</group>

execution -->
xsize="2048">

">(?:nc|ncat|nmap|nikto|sqlmap)</field>
ity tool executed: ${audit.executable}</description>
hive_medium</group>
```

4.3 TheHive Receiver Configuration

TheHive must be configured to accept incoming alerts from Wazuh and properly route them to the correct case templates. Configuration involves creating a dedicated organization for Wazuh alerts, configuring the case template with appropriate custom fields, and setting up the webhook receiver endpoint that parses incoming JSON payloads and creates cases with proper taxonomy and observable extraction.

4.3.1 TheHive Application Configuration

```
onf

r origin (if needed)
s = ["http://wazuh-manager:5601"]

wazuh alerts
source_ip, target_host, rule_id, mitre_technique

length for detailed alert descriptions

o extract from alerts
n", "hash", "filename", "url", "user-agent"]

o auto-analysis
ex-local", "url": "http://cortex:9001"}]

es from Wazuh
Total_GetReport_2_0",
P_Search_v2",
seIPDB_Query",
```

```
reatFox_IntelligenceSearch" ]
```

4.3.2 Integration Script (thehive.py)

Place this Python script in `/var/ossec/integrations/thehive.py` on the Wazuh manager server. The script receives alerts from Wazuh's integration framework, extracts relevant fields, constructs TheHive API requests, and creates cases with observables attached. Customize the severity mapping and field extraction logic to match your operational requirements.

```
.pt"""
```

```
TheHive details
```

```
/}" # From environment variable
```

```
TheHive severity (1-4)
```

```
ult
```

```
hive severity."""
```

```
items():
```

```
uh alert data."""
```

```
ert.get("srcip")
```

```
wazuh"]
```

```
ert.get("dstip")
```

```
", "wazuh"]
```

```
'wazuh"]
```

```
nd5_after") or data.get("syscheck.sha256_after")
```

```
ash) == 32 else "sha256"
```

```
wazuh"]
```

```
le from alert."""
```

```
t).get("description", "Unknown alert")
```

```
{}).get("name", "unknown-agent")
```

```
", " ")[19]
```

```
{agent_name} ({timestamp})"
```

```
ption with full alert context."""
```

```
.s
```

```
tion")}]}
```

```
live integration*
```

```
from Wazuh alert. """
```

```
THEHIVE_API_KEY}",  
"/json"
```

```
ert),  
escription(alert),  
ert.get("rule", {}).get("level", 0)),  
ated", f"rule-{alert.get("rule", {}).get("id")}"],  
vables(alert),
```

```
e",
```

```
et("_id")  
d case: {case_id}", file=sys.stderr)
```

```
{str(e)}", file=sys.stderr)
```

```
azuh integration framework.""")
    alert_json_file>", file=sys.stderr)

)

}], file=sys.stderr)
```

4.4 Automated Case Creation Workflow

The complete automated workflow from alert generation to analyst-ready case involves multiple stages of processing, enrichment, and routing. Understanding this flow helps troubleshoot integration issues and optimize the pipeline for reduced latency between alert occurrence and analyst notification. The typical end-to-end latency from Wazuh alert generation to TheHive case creation should be under 5 seconds under normal load conditions.

Stage	Component	Action	Duration	Error Handling
1. Detection	Wazuh Agent	Detects anomaly, generates alert locally	< 1 sec	Buffer locally if manager unreachable
2. Collection	Wazuh Manager	Receives alert, evaluates rules, assigns level	< 500ms	Queue for retry on failure
3. Filtering	Integration Config	Checks rule_id, level, group against filters	< 100ms	Drop non-matching alerts silently
4. Transformation	thehive.py	Maps fields, extracts observables, builds payload	< 100ms	Log error, continue next alert
5. Transmission	HTTP Client	POST to TheHive /api/case endpoint	< 2 sec	Retry with exponential backoff
6. Creation	TheHive Server	Validates payload, creates case, assigns ID	< 500ms	Return 400/500 with error details
7. Enrichment	Cortex (async)	Runs analyzers on observables (background)	10s - 5min	Queue for retry on timeout
8. Notification	TheHive	Sends notification to assigned analysts	< 5 sec	Queue if notification fails

4.5 Testing & Validation Procedures

After completing the integration configuration, execute the following validation procedures to confirm correct operation. Testing should be performed in a staging environment before production deployment, and re-executed after any configuration changes or software updates to prevent regressions.

Test Case	Procedure	Expected Result	Validation Command
API Connectivity	Test HTTP connectivity from Wazuh to TheHive	200 OK from /api/status	curl -s -o /dev/null -w "%{http_code}" http://thehive:9000/api/status

Authentication	Validate API key works for case creation	HTTP 201 with case ID	curl -X POST http://thehive:9000/api/case -H "Authorization: Bearer \$"
Alert Generation	Trigger test alert on Wazuh agent	Alert appears in Wazuh dashboard	Generate test event matching integrated rule
Case Creation	Verify case created in TheHive	New case visible with correct title	Check TheHive Cases tab for "[Wazuh]" prefix
Observable Extraction	Verify IOCs extracted correctly	Observables attached to case	Open case, check Observables tab
Cortex Enrichment	Confirm analyzers ran successfully	Reports attached to observables	Wait 2-5min, refresh observables
High Volume Load	Send 100 alerts/second for 5 minutes	No dropped alerts, <10s latency	Monitor Wazuh queues and TheHive response times
Error Handling	Stop TheHive, send test alert	Wazuh logs error, retries gracefully	Check /var/ossec/logs/integrations.log

PART V: SS7 Security Module (Telecom Signaling)

This section provides complete documentation for the SS7/Diameter security monitoring module, which is ****absolutely critical**** for a telecommunications operator SOC. The module monitors core network signaling protocols (SS7, SIGTRAN, Diameter) to detect attacks that cannot be seen at the IP layer alone, including subscriber location tracking, international revenue share fraud (IRSF), SMS interception, and signaling-based denial of service. Without this module, Djezzy's SOC is blind to the most damaging attack vectors targeting mobile network infrastructure.

C.1 Why SS7 Monitoring is Critical for Djezzy

SS7 (Signaling System No. 7) is the backbone protocol suite for all mobile networks globally. It handles every call setup, SMS delivery, subscriber roaming, and location update. Unlike IP-based attacks that can be detected by standard NSM tools, SS7 attacks operate at the core network layer and can cause catastrophic damage including mass privacy breaches, revenue loss in the millions of dollars, and complete service outages. For an Algerian telecom operator like Djezzy, SS7 monitoring is not optional-it is a regulatory requirement under ANRT (Autorite de Regulation de la Poste et des Communications electroniques) cybersecurity frameworks.

Threat Category	Impact on Djezzy	Detection Method	Regulatory Requirement
Location Tracking	Privacy breach, ANRT fines up to 2% revenue	SRV6/ATI rate analysis	ANRT Cybersecurity Law Art 12
Call/SMS Interception	Eavesdropping, corporate espionage	Forwarding pattern detection	Criminal Code Art 298-302
IRSF Fraud	\$10M+ annual loss potential (MENA avg)	International call pattern analysis	ANRT Fraud Prevention Guidelines
Signaling DoS	Network outage, 100K+ subscribers affected	Message flood detection	QoS Regulations Art 45
SIM Cloning/Fraud	Identity theft, billing fraud	UL storm & IMEI mismatch	Consumer Protection Law
USSD Service Abuse	Revenue loss, service degradation	USSD rate limiting	Value-Added Services Regs
Roaming Partner Compromise	Supply chain attack, data exfiltration	Partner anomaly detection	Roaming Agreement Compliance

C.2 SS7 Module Architecture

The SS7 security module consists of three microservices that integrate seamlessly with the existing 39-service architecture. The design follows the same patterns as other security tools: capture via dedicated collector, analyze via rules engine with Kafka buffering, and output alerts to both Wazuh SIEM and TheHive case management. All services are containerized, orchestrated via Docker Compose/Kubernetes, and monitored through Prometheus/Grafana.

Service	Container Name	Purpose	Ports	Resources	Network
SS7 Collector	djezzy-ss7-collector	Capture M3UA/SCTP/Diameter from 2974-2905, 3868, 7000	2974-2905, 3868, 7000	8 CPU / 16GB RAM	soc-events
SS7 Analyzer	djezzy-ss7-analyzer	Real-time attack detection with YARA rules engine	8000	6 CPU / 12GB RAM	soc-backend + soc-events
Diameter Monitor	djezzy-diameter-monitor	LTE/EPS Diameter interface monitoring (3GPP S6a)	3868 (S6a-Gx)	2 CPU / 4GB RAM	soc-events

C.3 Detection Rules Inventory (17 Rules)

The SS7 analyzer includes 17 built-in detection rules organized into four categories: Location Tracking/Privacy (5 rules), Fraud Detection (6 rules), Network Attacks/DoS (6 rules). Each rule is defined as a YAML file in `config/ss7/rules/` and supports complex conditions with AND/OR/NOT logic, field comparisons, sliding window rate calculations, and MITRE ATT&CK; mapping. Rules can be enabled/disabled independently via API or file modification without service restart.

Category	Rule Count	Key Rules	Primary Target Protocols
Location Tracking & Privacy	5	LOCATION_TRACKING_SUSPICIOUS, SRI_FROM_UNUSUAL_SOURCE, PROXIMITY_NUMBER_ANOMALY	MAPS, SOURCE, PROXIMITY
Fraud Detection	6	IRSF_PATTERN_DETECTED, WANGIRI_FRAUD_PATTERNS, SUSPICIOUS_FORCE, SMS_FORWARDING	IRSF, WANGIRI, SMS, FORWARDING
Network Attacks & DoS	6	SIGNALING_DOS_DETECTED, GT_TRANSLATION_ATTACK, MALFORMED_MESSAGE, UNUSUAL	SIGNALING, GT, MALFORMED, UNUSUAL

C.4 Integration with Existing Stack

Downstream System	Integration Method	Data Format	Use Case
Wazuh SIEM	Kafka consumer + custom decoder	JSON alert + Wazuh rule XML (integration with Wazuh rules engine)	Consolidate SS7 events with other security events, unified dashboard
TheHive Case Mgmt	REST API (thehive4py client)	JSON alert -> TheHive case with observability	Automated case creation for HIGH/CRITICAL, analyst workflow
Grafana Dashboards	Prometheus metrics + Elasticsearch	Time series alert tables	Real-time SS7 security overview dashboard
Kafka Event Bus	Producer/Consumer on dedicated topic	Avro/JSON schema (config/ss7/kafka/topics)	Topic buffering, replay capability, multi-consumer
Elasticsearch	Index template (ss7-*)	Structured JSON documents	Long-term retention, search, investigation
Prometheus Metrics	/metrics endpoint (prometheus client)	Counter/Gauge/Histogram	Service health, message rates, alert counts
External Fraud System	Kafka producer to ss7-fraud-indicators topic	JSON fraud indicators	Feed IRSF/premium-rate detection to existing FMS

C.5 Quick Reference Commands

```
curl -s http://localhost:7000/health | jq .
curl -s http://localhost:8000/health | jq .
docker compose -f docker-compose.prod.yml ps | grep ss7
docker compose -f docker-compose.prod.yml logs -f --tail=100 ss7-collector
curl -s http://localhost:8000/api/v1/stats | jq .
curl -X POST http://localhost:8000/api/v1/rules/reload
curl -s http://localhost:8000/api/v1/rules | jq ".[] | {name, severity, enabled}"
curl -X POST http://localhost:7000/api/v1/message/inject -d @test_ss7.json
kafka-topics.sh --list --bootstrap-server localhost:9092 | grep ss7
kafka-console-consumer --topic ss7-alerts --from-beginning --max-messages 5
ls -la /var/lib/docker/volumes/ss7_pcaps/_data/
curl -s -u admin:admin "http://grafana:3000/api/dashboards/import" -d @ss7-dashboard.json
```

Appendix A: Port Mapping Reference

Complete port reference for all 39 services in the SOC platform. Use this table when configuring firewall rules, network policies, or troubleshooting connectivity issues between services. Internal ports are used for inter-service communication within Docker networks; external ports are exposed to the host for access from outside the container environment or from other network segments.

Service	Internal Port	External Port	Protocol	Network	Purpose
PostgreSQL	5432	5432	TCP	soc-backend	Primary database
PgBouncer	6432	6432	TCP	soc-backend	Connection pooling
Redis Master	6379	6379	TCP	soc-backend	Cache/session store
Redis Sentinel	26379	26379	TCP	soc-backend	Redis HA monitoring
Elasticsearch	9200	9200	TCP	soc-backend	Search/index API
ES Transport	9300	-	TCP	soc-backend	Inter-node communication
Kibana	5601	5601	TCP	soc-frontend	Visualization UI
Kafka Broker	9092	9092	TCP	soc-events	Message broker clients
Kafka Internal	9093	-	TCP	soc-events	Inter-broker communication
Zookeeper	2181	2181	TCP	soc-events	Kafka coordination
Wazuh Manager	1514	1514	UDP/TCP	soc-backend	Agent data ingestion
Wazuh API	55000	55000	TCP	soc-frontend	REST API
Wazuh Indexer	9200	-	TCP	soc-backend	Wazuh ES backend
GRR Server	8000	8000	TCP	soc-backend	EDR administration
GRR Agent	8080	-	TCP	soc-backend	Agent communication
Suricata	7736	7736	TCP	soc-events	EVE JSON output
Suricata Stats	7737	7737	TCP	soc-events	Statistics socket
Zeek Controller	50000	-	TCP	soc-events	Management interface
Arkime Viewer	8005	8005	TCP	soc-frontend	PCAP viewer UI
Arkime Capture	-	-	TCP	soc-events	Packet capture (transparent)
TheHive	9000	9000	TCP	soc-frontend	Case management UI/API
Cortex	9001	9001	TCP	soc-backend	Analyzer engine API
MISP	443	443	TCP	soc-frontend	Threat intelligence platform
OpenCTI	8080	8080	TCP	soc-frontend	Threat intelligence (STIX)
OpenVAS/GVM	9392	9392	TCP	soc-backend	Vulnerability scanner
DefectDojo	8080	8080	TCP	soc-frontend	Vulnerability management
Prometheus	9090	9090	TCP	soc-monitoring	Metrics collection
Grafana	3000	3000	TCP	soc-frontend	Dashboards/alerting
Alertmanager	9093	9093	TCP	soc-monitoring	Alert routing/deduplication
Kong Admin	8001	8001	TCP	soc-frontend	Gateway administration
Kong Proxy	8000	80/443	TCP	soc-frontend	API proxy (public)
Nginx	80/443	80/443	TCP	soc-frontend	Reverse proxy (public)
SOC Platform	3000	-	TCP	soc-frontend	Next.js application

Appendix B: Troubleshooting Guide

This troubleshooting guide addresses common issues encountered during deployment and operation of the Djezzy SOC Platform. Each issue includes symptoms, diagnostic steps, and recommended resolutions. For issues not covered here, consult the component-specific documentation or escalate to the platform engineering team with relevant logs collected.

Issue	Symptoms	Diagnostic Steps	Resolution
Service fails to start	Container exits immediately, Exit code non-zero	Check logs <service>, Check .env variables	Verify permissions, correct UID
Database connection refused	Cannot connect to PostgreSQL	docker port -p 5432 postgresql, Check pg_isready	Verify Postgres is running, Check pg_hba.conf
Kafka consumer lag increasing	Alerts delayed, Events backlog growing	docker exec kafka-topics --describe, Check broker health	Scale consumer group, Increase replication factor
Elasticsearch cluster red	Some indices unavailable, Search failures	curl -XGET localhost:9200/_cat/allocation?v, Check disk usage	Rebalance cluster, Free disk space, _cluster/reroute
Wazuh agents disconnected	Agents show "Never connected"	List agents in Wazuh UI, Check agent.log	Restart agent, Verify firewall rules, Check 1514/1515 port access
TheHive cases not created	Wazuh alerts present but no cases in TheHive	Check thehive.log, Test API key manually	Update TheHive API key, Restart wazuh-manager
High memory usage	OOM kills, System swapping	docker stats, free -h, Check JVM heap settings	Adjust container limits, Tune JVM -Xmx, Scale horizontally
SSL/TLS certificate errors	HTTPS connections failing, Certificate warnings	docker exec -t openssl s_client -connect host:port, Check logs	Renew certificates, Update truststore, Regenerate with correct S
Kong gateway 502/504	Upstream errors, Timeout responses	Check logs, Check upstream health, Verify DNS	Restart upstream service, Adjust Kong timeouts
Grafana dashboards empty	No data displayed, Panel errors	Check Prometheus datasource, Verify queries	Fix datasource config, Update templates, Check time range